

Demystifying Memorization in LLM-Based Program Repair via a General Hypothesis Testing Framework

JIAOLONG KONG, Singapore Management University, Singapore

XIAOFEI XIE*, Singapore Management University, Singapore

SHANGQING LIU*, Nanjing University, China

Large Language Models (LLMs) have achieved remarkable success in various applications, particularly in code-related tasks such as code generation and program repair, setting new performance benchmarks. However, the extensive use of large training corpora raises concerns about whether these achievements stem from genuine understanding or mere memorization of training data—a question often overlooked in current research. This paper aims to study the memorization issue within LLM-based program repair by investigating whether the correct patches generated by LLMs are the result of memorization. The key challenge lies in the absence of ground truth for confirming memorization, leading to various ad-hoc methods designed for its detection. To address this challenge, we first propose a general framework that formalizes memorization detection as a general hypothesis testing problem, where existing approaches can be unified by defining a *low-probability event* under the *null hypothesis* that the data is not memorized. The occurrence of such an event leads to the rejection of the null hypothesis, indicating potential memorization.

Based on this framework, we design two specific methods (i.e., low-probability events) to detect potential memorization: 1) basic ground-truth matching, and 2) reassessment after substantial code mutation. We investigate the memorization issue in LLM-based program repair using two datasets: Defects4J, a widely used benchmark that is likely included in the training data, and GitBug-Java, a new dataset that is unlikely to be part of the training data. Our findings reveal that a significant portion of correct patches exactly match the ground truths in Defects4J (e.g., 78.83% and 87.42% on GPT-3.5 and CodeLlama-7b, respectively). Moreover, even after significant modifications to the buggy code, where the original repairs should not be generated, a considerable percentage of bugs (e.g., 81.82% on GPT-3.5 and 88.24% on CodeLlama-7b) continue to be fixed exactly as in the original bug fixes, indicating a high likelihood of memorization. Furthermore, we evaluate existing memorization detection methods and demonstrate their ineffectiveness in this context (e.g., most AUROCs are below 0.5). The theoretical analysis under our hypothesis testing framework shows that their defined events may not meet the requirements for being low-probability. The study highlights the critical need for more robust and rigorous evaluations in LLM-based software engineering research, ensuring a clear distinction between true problem-solving capabilities and mere memorization.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Program Repair, Code Memorization

*Corresponding authors.

Authors' Contact Information: [Jiaolong Kong](mailto:jlkong@smu.edu.sg), Singapore Management University, Singapore, Singapore, jl原因@smu.edu.sg; [Xiaofei Xie](mailto:xfxie@smu.edu.sg), Singapore Management University, Singapore, Singapore, xfxie@smu.edu.sg; [Shangqing Liu](mailto:shangqingliu@nju.edu.cn), Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, shangqingliu@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2025/7-ARTFSE120

<https://doi.org/10.1145/3729390>

ACM Reference Format:

Jiaolong Kong, Xiaofei Xie, and Shangqing Liu. 2025. Demystifying Memorization in LLM-Based Program Repair via a General Hypothesis Testing Framework. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE120 (July 2025), 23 pages. <https://doi.org/10.1145/3729390>

1 Introduction

In recent years, Large Language Models (LLMs) have experienced rapid development, significantly advancing the landscape of artificial intelligence. These models have been successfully applied across a wide range of applications, such as text summarization and translation. The exceptional performance of LLMs can largely be attributed to their training methodologies, which involve pretraining on massive amounts of data. For instance, models like GPT-4 are trained on about 45 TB of data [3], encompassing books, websites, and other text, enabling them to generate human-like responses across various domains.

Meanwhile, LLMs have been extensively applied to complex code tasks, transforming traditional approaches in software engineering (SE). These models excel in a variety of applications including code generation [17, 31, 37, 45], vulnerability detection [38, 44, 57], code review [21], code completion [22], commit message generation [33], program specification generation [34] and program repair [18, 25, 28, 52]. Many models, such as ChatGPT, Codex, CodeLlama, and CodeQwen, have demonstrated superior performance, often achieving state-of-the-art results. The adoption of LLMs in SE tasks signifies a revolutionary shift, enhancing the performance of many tasks compared to traditional methods. For example, in the realm of code review, LLM-based method can achieve an Exact Match score of 19.5% on the benchmark CRN [21] while the traditional method CodeReviewer [30] only reaches 14.8%. In automated program repair, the LLM-based method can successfully repair 162 bugs compared to 76 bugs fixed with the traditional method [32].

While the remarkable advancements of LLM-based methods in SE are impressive, they raise a critical question: *Do these LLMs genuinely acquire program understanding and code reasoning capabilities, or do they merely perform well because they have seen these code during training with massive data corpora?* Many widely-used evaluation benchmarks, such as Defects4J [27], may have been included in their training datasets. For instance, existing studies [10, 12, 53] suggest that language models can memorize their training data, which could skew performance metrics. Understanding the extent to which performance improvements are driven by true learning versus simple memorization is crucial, especially when many LLM-based methods have been proposed in SE community. This distinction is significant not only for better assessing the real capabilities of LLMs or these LLM-based methods but also for identifying opportunities for further enhancements in SE research.

In this paper, we aim to delve into the memorization issue within the realm of automated program repair. APR has been widely studied in recent years, with numerous techniques [8, 26, 28, 46, 49, 51, 52, 55, 56] being introduced. Although LLM-based methods have significantly surpassed traditional approaches in this field, the investigation into code memorization by these models remains scant. For example, Fig. 1 illustrates a prompt used for repairing the bug Closure-36 in the Defects4J dataset alongside the response generated by GPT-3.5. The response message is structured into two parts: a description of the reasoning behind the error and a generated patch (highlighted in green). In this instance, the patch is exactly the same as the one provided by the developers. However, the callee method `getSingletonGetterClassName` mentioned in the patch does not exist in the prompt, indicating that the LLM should not have prior knowledge of it. This discrepancy suggests that this repair may primarily result from memorization rather than true understanding.

A fundamental challenge is to confirm whether a generated fix results from memorization. Currently, there is even no standard definition of memorization. A common approach is to check

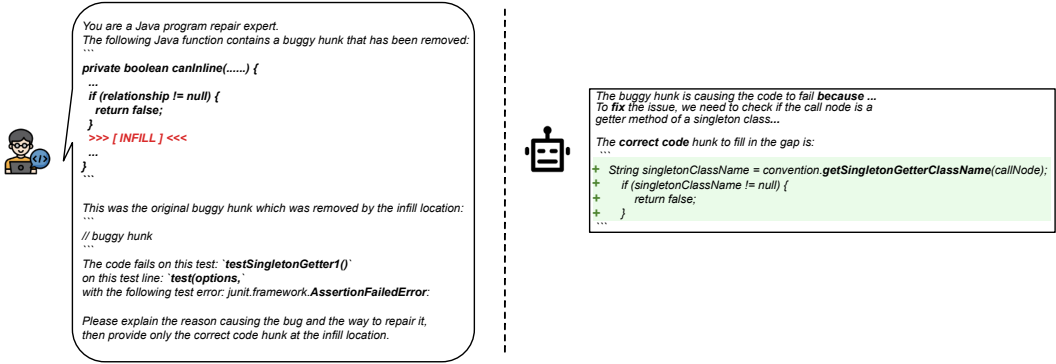


Fig. 1. An motivation example of Closure-36 in Defects4J

whether the data appears in the training set [10–12, 53], essentially determining if the model has encountered it before. However, there are two main concerns with this strategy: 1) the presence of data in the training set does not guarantee that the LLM has memorized it, and 2) for closed-source LLMs (e.g., ChatGPT), we often lack access to their training data, making it difficult to check the training data directly. Therefore, it is crucial to develop methods to detect potential memorization in program repair and other tasks.

While some studies [15, 19, 20, 29, 41] have paid attention to memorization issues, they often adopt different, ad-hoc strategies due to the lack of ground truth for determining memorization. To tackle this problem, we first propose a general memorization detection framework based on *hypothesis testing theory* [5, 48], which unifies existing ad-methods in a framework. Specifically, to detect memorization, hypothesis testing starts with a null hypothesis (H_0), which states that *the model has not memorized the data*. The goal is to identify a low-probability event [9, 36] that should not occur if the null hypothesis is true. The occurrence of such an event leads to the rejection of the null hypothesis, supporting the alternative hypothesis that the data is likely memorized. Under this unified framework, the goal of existing work can be defined as designing various low-probability events for rejecting the null hypothesis and indicate potential memorization, such as checking if the response is identical to a training example [10–12, 53] or whether the model’s confidence is unusually high [29, 39].

Building on the hypothesis testing framework, we designed specific low-probability events to detect potential memorization in LLM-based program repair¹. Inspired by educational practices, where teachers assess whether students truly understand concepts rather than merely memorizing solutions, we propose two such events. The first method compares the generated repair with the ground truth. If the LLM’s output exactly matches the standard solution, it may indicate potential memorization, as the probability of an exact match should be relatively low under the null hypothesis (i.e., no memorization). The second method involves mutating the buggy code by altering critical program contexts or renaming key identifiers and then reassessing the LLM’s output. If the LLM continues to produce the original repair despite these substantial changes, it suggests reliance on memorized solutions, as this behavior would constitute a low-probability event, or even 0, under the null hypothesis.

Based on our proposed framework, we further investigate the extent of code memorization in LLM-based APR methods. Specifically, we selected two representative APR methods for this study: the state-of-the-art tool, ChatRepair [52], and a memorization-directed prompting strategy, MemPrompt [20]. ChatRepair was chosen for its strong ability to repair a large number of bugs,

¹While this paper primarily focuses on the APR domain, our framework is general and could be extended to other tasks.

providing enough cases to explore whether these repairs result from genuine understanding or memorization. MemPrompt, on the other hand, employs simple prompts that do not include detailed feedback, such as runtime traceback logs, which are used in ChatRepair. As a result, repairs in MemPrompt are more challenging, and occurrences of exact matches in MemPrompt tend to provide stronger evidence of memorization. To demystify the memorization and evaluate our proposed low-probability events, we selected two datasets: Defects4J, which is widely used and thus more likely to be memorized, and GitBug-Java, a new dataset that is unlikely to be part of the training data and therefore less prone to memorization. We conducted the experiments on two representative LLMs, i.e., ChatGPT-3.5 and CodeLlama-7b.

Our experimental results show that ChatRepair, utilizing ChatGPT-3.5 and CodeLlama, successfully repaired 59 and 50 bugs (with a temperature of 1.0) out of 154 single-hunk scenarios in Defects4J. Notably, 50 out of 59 (84.75%) and 39 out of 50 (78.00%) of these repairs exactly matched the ground-truth fixes provided by developers, suggesting a high likelihood of memorization. To confirm this, we applied the second method: omitting the surrounding context of the buggy code or renaming variables within the buggy functions. Surprisingly, both ChatGPT-3.5 and CodeLlama can still produce the same patches, even when key context or identifiers were removed—events that should be highly unlikely without memorization. For example, even after removing necessary context, MemPrompt still generated 19 out of 23 (82.61%) and 15 out of 17 (88.24%) identical patches. However, we found that the memorization issue significantly decreased on the new dataset GitBug-Java, further validating our methods in distinguishing memorization from non-memorization cases.

We also evaluated the effectiveness of existing memorization detection methods, including Perplexity [13, 29], Min-K % [41], CDD [15], PPL-zlib ratio [12], and PPL-PPL ratio [53] in the APR context. Our analysis showed that these methods are not yet effective in APR scenarios, as most AUROC values were below 0.5. Further theoretical analysis shows that the events they defined are not true low-probability events (e.g., 47.5%+), which explains why they are ineffective at identifying memorization in APR.

To summarize, this paper makes the following contributions:

- We are the first, to the best of our knowledge, to systematically investigate the potential memorization issues within existing LLM-based APR methods.
- We propose a general memorization detection framework based on hypothesis testing, and unify existing works in this framework.
- We propose two strategies for identifying potential cases of memorization in APR domain: basic ground-truth matching and a mutation-based strategy.
- We conduct comprehensive experiments to demystify memorization in LLM-based APR and evaluate the effectiveness of state-of-the-art memorization detection methods. Our findings² provide valuable insights for future APR research and related software engineering tasks that rely on LLMs, highlighting the critical need for memorization-aware evaluation and detection strategies.

2 Background

2.1 LLM-Based Program Repair

Recent work in Automated Program Repair (APR) has focused on utilizing Large Language Models (LLMs) to generate patches directly. This LLM-based APR approach starts by constructing a prompt with the original buggy code and then prompts the LLM to either fill in the correct code at the bug's location (cloze-style APR) or generate an entirely new code snippet for the patch. This approach

²All experimental details are available on the website [4].

adheres to the traditional Generate and Validate (G&V) repair paradigm, where numerous patches are generated by sampling from the same initial prompt, followed by the validation of each patch. A further manual check is conducted for earlier patches that pass all tests (i.e., plausible patches) to ensure their correctness. To enhance the effectiveness of the prompts, LLM-based APR tools often incorporate additional information to help the LLM understand the underlying cause of the bugs. These methods can be broadly classified into two groups based on whether they involve fine-tuning the LLM, with both approaches demonstrating strong performance. There are some typical works of fine-tuning for APR [23, 42, 50] and some other works interact with LLMs in prompt [28, 52]. Compared with these works, which propose new techniques to improve repair accuracy, we aim to explore the potential memorization issue in LLM-based program repair.

2.2 Code Memorization in LLM

The impressive performance of large language models (LLMs) is typically attributed to their substantial size and the wide variety of training data, enabling them to retain vast information from the training set. While this memorization capability (i.e., verbatim memorization [10]) can enhance the model's output when encountering similar inputs from the training data, it also poses potential security risks, such as user privacy and confidentiality exposure. Due to potential security risks of memorization that may arise when using large models, research on memorization is gradually receiving attention [12, 15, 19, 20, 24, 29, 41, 53]. Many of these studies aim to extract memorization data [12, 41] by developing various metrics [29] for estimating the extent of memorization. However, while much of this research concentrates on general memorization, such as natural language conversations, there has been less focus on code scenarios. A recent study by Yang et al. [53] has designed an investigation into code memorization in pre-trained models, including Codeparrot, PolyCoder, and GPT-Neo. However, they need to access the training data for confirming the memorization. Access to pre-training data and state-of-the-art models, such as GPT-3.5 and GPT-4, is usually restricted in practical settings.

3 Methodology

In the context of APR task, demonstrating that a correct repair results from code memorization is highly challenging. This difficulty arises from two key factors: 1) The lack of transparency regarding the LLM's training data, which prevents us from determining whether the LLM has encountered the code pair during training process; 2) The absence of a standard definition or ground truth for code memorization. Although many existing works discuss memorization, there is no widely accepted definition. Actually, it is hard to define memorization. For instance, the presence of a solution in the training dataset does not guarantee its memorization. Even if the LLM has encountered the correct solution in the training data, it may not reproduce it identically when queried.

To this end, we first investigate existing memorization detection methods and introduce a general hypothesis testing-based framework [5, 48] to formalize the memorization issue.

3.1 A Hypothesis Testing Framework for Memorization Detection

We have thoroughly examined existing works on memorization detection [13, 15, 29, 35, 41] and found that they employ very different approaches to confirm memorization. To formalize the problem, we propose a general hypothesis-testing framework that can integrate various existing methods into a unified structure.

Defining the Hypotheses. Given a code pair (b, p) , where b represents the buggy version and p the patched version generated from the LLMs, we define the hypotheses as follows:

- Null Hypothesis (H_0): *The model has not memorized the code repair for b .*

Table 1. Description with details for existing methods.

Existing Method	Low-Probability Events Under the Null Hypothesis (i.e., Data Not Memorized)	Applicability
Perplexity [13, 29]	The output is generated with exceptionally high confidence.	✓
Min-K% [41]	There are few outlier words, and the model makes all predictions with high confidence.	✓
CDD [15]	Similar or even identical outputs are generated when the model is queried with the same input multiple times.	✓
Neighbourhood Comparison [35]	The loss value of the data is substantially lower than that of its neighbors after minor perturbations.	✗
PPL-zlib ratio [12]	The prediction confidence is exceptionally high even if the output contains high-entropy data (e.g., user name and addresses).	✓
PPL-PPL ratio [53]	For the same input, the perplexity of a large model is significantly smaller than that of a small model.	✓
Exact match in training set [10, 12]	The output exactly matches the data in the training set.	✗

- Alternative Hypothesis (H_1): *The model has memorized the repair of b .*

The next step in hypothesis testing often involves designing a test statistic that evaluates the plausibility of the null hypothesis. Typically, a predefined threshold is used to decide whether to reject the null hypothesis, with a common significance level of 0.05. However, it can be challenging to directly design a measurable test statistic (i.e., a *sufficient* condition) for proving memorization.

Low-Probability Event. Instead of seeking such a sufficient condition directly, we can analyze memorization by identifying a *necessary* condition ($\neg e$) such that $H_0 \rightarrow \neg e$. In other words, under H_0 , the event e cannot occur (e.g., with probability 0). Based on *Modus Tollens rule* [6, 7, 40], if e is observed (i.e., $\neg e$ is false), then H_0 must be rejected (false), implying H_1 . In practice, however, it is difficult to identify events that have strict 0-probability under H_0 . Consequently, we propose designing *low-probability events* e as a proxy. While such events cannot definitively prove that H_0 is false, their occurrence increases the likelihood that H_0 to be incorrect.

Theoretical Analysis of the Low-Probability Event. The requirement of low-probability event can be derived using Bayes' Theorem. Specifically, our goal is to find an event e that increases the probability $P(H_1)$ upon its observation, i.e., a large $P(H_1|e)$. Bayes' Theorem tells us:

$$P(H_1|e) = \frac{P(e|H_1) * P(H_1)}{P(e)}$$

By the Law of Total Probability, we can express $P(e)$ as $P(e|H_1)P(H_1) + P(e|\neg H_1)P(\neg H_1)$. As $\neg H_1$ is equal to H_0 , we have:

$$P(H_1|e) = \frac{P(e|H_1) * P(H_1)}{P(e|H_1)P(H_1) + P(e|\neg H_1)P(\neg H_1)} = \frac{P(e|H_1) * P(H_1)}{P(e|H_1)P(H_1) + P(e|H_0)P(H_0)}$$

Here, $P(H_1)$ and $P(H_0)$ can be viewed as prior probabilities and are treated as constants. Thus, to boost $P(H_1|e)$, we can (1) *increase* $P(e|H_1)$ and (2) *decrease* $P(e|H_0)$. In practice, it is easier to ensure $P(e|H_1)$ is fairly large, so keeping $P(e|H_0)$ *very small* is especially critical. For example, if $P(e|H_0)$ is close to zero, then regardless of the value of $P(e|H_1)$ is, $P(H_1|e)$ approaches 1, providing strong evidence for memorization.

Criteria for Determining Memorization. Based on the above analysis, observing a low-probability event e under H_0 suggests the possibility of memorization, although the strength and reliability of this inference depend on the probability of e . In theory, *the smaller $P(e|H_0)$ is, the stronger the indication of potential memorization, thereby supporting rejection of H_0 .* In practice, one can define a small threshold θ such that $P(e|H_0) < \theta$, thereby designating e as a low-probability event.

3.2 Unifying Existing Methods within Hypothesis Testing Framework

While many methods for detecting memorization have been proposed, they vary significantly, making it difficult to understand their underlying principles and why each works. In this section, we unify these diverse methods within our framework: essentially, their goal is to define a low-probability event under the null hypothesis (i.e., low $P(e|H_0)$). As summarized in Table 1, we explain how each method can be interpreted as identifying a low-probability event. The column *Applicability* indicates whether the method is applicable to our repair tasks.

Perplexity [13, 29]: Perplexity determines memorization by measuring the model's confidence in its own predictions. A lower perplexity value indicates higher confidence, suggesting that the model is more familiar with the data. Intuitively, a model tends to show greater confidence on examples it has encountered during training. Therefore, a lower perplexity on a given data point may imply that the model is retrieving memorized information.

Low-Probability event: If the data has not been memorized, the probability of *the model predicting the target data with exceptionally high confidence* (i.e., achieving low perplexity) should be low.

Min-K% [41]: To determine whether an input was present in the pretraining data of an LLM, this method assumes that an unseen example is likely to contain a few outlier words with low probabilities (i.e., the average value of the K% tokens with the lowest probabilities) under the LLM, whereas a seen example is less likely to have such words with low probabilities.

Low-Probability event: If the data has not been memorized, the probability of *achieving a high Min-K% value* (i.e., with few outlier words) is low.

CDD [15]: The method assumes that memorization leads to a more peaked output distribution. If an LLM consistently generates identical or highly similar texts through sampling, it is likely due to memorization. Specifically, CDD samples multiple outputs for a given input and checks whether these outputs are very similar or even identical.

Low-Probability event: If the data has not been memorized, the probability of *achieving a peaked output distribution after sampling multiple outputs with the same input* (i.e., resulting in similar or identical outputs) is low.

Neighbourhood Comparison [35]: When given a target sample, this method first crafts neighbors through minimal changes that fully preserve the semantics. Comparing the losses of both target sample and its crafted neighbours, if target sample's loss value being substantially lower than those of the neighbours, this could only be a result of overfitting and, therefore, the target sample is a training member. Note that this method cannot be directly applied to the APR task, as the neighbor-finding technique in their approach is designed specifically for textual data. Additionally, obtaining loss values may not always be feasible, particularly for closed models like ChatGPT.

Low-Probability event: If the data has not been memorized, the probability of *achieving a significantly lower loss value for the input compared to its neighbors* is low.

PPL-zlib ratio [12]: zlib [2] is used to calculate the zlib entropy of a text. Specifically, higher entropy indicates that the sample is more likely to include random data, such as encrypted text or privacy-related information (e.g., user IDs). Such data appears in the output because it is likely to have already appeared in the training set. This method computes the ratio of model perplexity to zlib entropy. A small ratio suggests that the output is more likely to be memorized.

Low-Probability event: If the data has not been memorized, the probability of *achieving high-confidence predictions for outputs containing high-entropy data* (e.g., addresses) is low.

PPL-PPL ratio [53]: It aims to detect non-trivial memorization, which is slightly different from our null hypothesis. The assumption is that large models have a greater capacity for memorization compared to smaller models. This method uses the ratio of perplexity between a large model and a small model for the same input. A small ratio indicates that the output is more likely to contain non-trivial memorization, as the perplexity of the large model is significantly lower than that of the small model.

Low-Probability event: If the data does not involve (non-trivial) memorization, the probability that *the large model's perplexity is much lower than that of the small model* is low (i.e., they are expected to have relatively similar perplexity).

Exact match in training set [10, 12]: One commonly used method is to match the target against the training set. Intuitively, if identical fragments are found in the training set, the sample is considered memorized. This method is not applicable in cases where the training dataset is unavailable.

Low-Probability event: If the data has not been memorized, the probability of *finding one or more exact matching samples in the training set* is low.

As discussed, while many different methods have been proposed based on some intuition or hypothesis, it remains unclear whether their events are truly low-probability, which is crucial for confidently determining memorization (see Section 3.1). For example, in the case of Perplexity, high confidence does not always indicate memorization, and non-memorized data can also achieve high confidence, thereby increasing the probability of that event. Similarly, inclusion in the training data does not guarantee that the data has been memorized. Our experiments indicate that these methods are not effective because they tend not to represent true low-probability events in some cases (see results in Section 5.3).

3.3 Designing Low-Probability Events in LLM-Based Program Repair

Based on the framework, we need to design the specific low-probability events under H_0 for determining the potential memorization issues in LLM-based program repair. Drawing inspiration from educational methods, where teachers often assess whether students truly understand concepts rather than simply memorize them, we have designed two such events to uncover memorization issues in LLMs. Specifically, student assessment methods involve various approaches such as comparing student answers to textbook solutions and retesting students on altered questions they previously answered correctly. We design the two events as:

3.3.1 Ground-truth Matching. Teaching analogy: we consider the scenario where a student's answer is not only correct but also exactly mirrors the solution presented in the textbook, including the style of description. Such a scenario could suggest that the answer may have been memorized rather than truly understood.

In the context of LLMs and APR, one straightforward method is to compare the fixes generated by the LLM with the ground-truth fixes provided by developers. While there can be multiple patches that can repair a bug, an exact match with the developer-provided solution increases the likelihood that the patch is memorized. Formally, this constitutes a low-probability event under H_0 : assuming the patch (b, p) is not memorized by the LLM, the probability of generating patched code p that exactly matches the ground truth would be small.

3.3.2 Retest on Mutated Buggy Code. Teaching analogy: to assess a student's genuine understanding of a concept, a teacher often modifies questions that have been previously answered correctly, rather than reusing them unchanged. This tests the student's understanding and ability to apply their knowledge. Similarly, in the context of LLM-based repair, for the correct fixes, we modify the buggy code and observe how the LLM responds. If the LLM still produces patches identical to the previous ones, it increases the suspicion that these fixes are memorized.

Specifically, we propose three mutation strategies on the original buggy code, as detailed below

- **Removing Necessary Context.** We identify essential variables from the buggy code based on the ground-truth patch, which include all variables involved in the patch. We then mask all statements directly related to these variables (named *Mask*), such as variable definitions and their dependent uses, and ask the LLM to repair the modified code. An extreme version of this mutation involves removing all statements preceding the fix location (named *Omit*). The hypothesis is that, without the necessary context, the LLM cannot produce the same fix with previous one.
- **Renaming Variable Names.** After identifying all crucial variables in the fixes, we rename these variables with arbitrary, nonsensical names. We hypothesize that if the LLM understands the code semantics, it should adapt the repair to accommodate the new variable names, rather than reuse the old names.

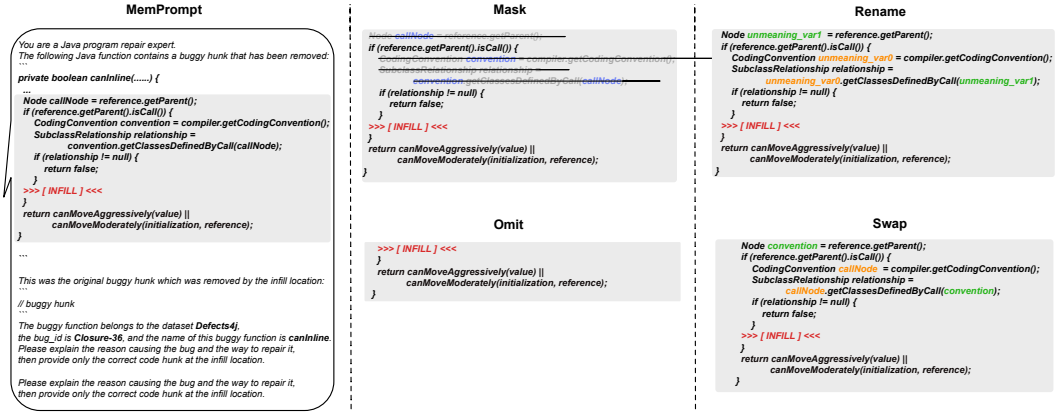


Fig. 2. Example of the mutations in Closure-36

- **Swapping Variable Names.** This strategy involves swapping the names of two crucial variables, which could confuse the LLMs if they rely on rote memorization. The hypothesis is: a genuine understanding would enable the LLM to correctly adjust the patch to reflect swapped names.

Within the context of our hypothesis testing framework, this method constructs a particularly low-probability event under H_0 : assuming the patch (b, p) is not memorized by the LLM, after we substantially mutate the buggy code from b to b' , the LLM should not be able to reproduce the same patch p (i.e., with 0 or extremely low probability) because key context or variable names have been removed or altered. Instead, it should generate a very different patch p' that aligns with these significant mutations.

Example. Fig. 2 demonstrates the application of four distinct prompts to the bug Closure-36. To generate the Mask prompt, we first determined which statements from the correct patch should be excluded. The correct patch, depicted in the green section of Fig. 1, contains three critical identifiers: `singletonClassName`, `convention`, and `callNode`. Given that `singletonClassName` is newly declared and absent in the original buggy function, our focus was on the lines that include `convention` and `callNode`. These lines, highlighted in blue, are removed from the buggy code snippet to create the Mask prompt. Further refining this approach, the Omit prompt, is constructed by discarding all the prefix surrounding the buggy code snippet. The right part illustrates the prompts generated using the two variable name mutation rules. We initially identified the relevant variable names from the correct patch, as shown in Figure 1: `convention` and `callNode`. For the Renaming Variable Names strategy, we replace every occurrence of `convention` in the original buggy code with `unmeaning_var0` (highlighted in orange) and `callNode` with `unmeaning_var1` (highlighted in green). In a similar vein, the Swapping Variable Names prompt is completed by swapping all occurrences of `convention` with `callNode`.

If the patch generated after the mutation remains the same as the patch before the mutation (i.e., still using elements like `convention` and `callNode` that have been removed or renamed), it demonstrates a high likelihood of memorization.

4 A Study on Memorization in LLM-Based Program Repair

Building on the framework and designed detection methods, we further investigate memorization issues in LLM-based APR. To conduct this study, we need to select specific LLMs, APR methods, and benchmarks.

Specifically, we chose two widely used models: GPT-3.5 and CodeLlama-7b-Instruct. For APR methods, we selected ChatRepair, a state-of-the-art LLM-based APR tool that generates informative prompts (e.g., error feedback) for program repair. Note that ChatRepair is a multi-iteration method. In our study, we limited it to a single iteration and renamed it to **ChatRepair-Single**. The rationale

behind using a simple single-iteration approach in ChatRepair is to strengthen the evidence for memorization. If events such as exact matches occur across multiple iterations, it becomes hard to determine whether they result from memorization or from improved reasoning due to the additional feedback provided in multiple iterations, thereby reducing the confidence in detecting memorization.

Additionally, we developed another method (MemPrompt) for comparison, which removes the informative prompts from ChatRepair, making it less likely to generate the correct patch (i.e., a lower probability event). However, removing such information might cause the model to fail in generating any correct patches, which could make it difficult to determine whether there is no memorization or if the prompt is simply too weak. To mitigate this, we include some project-related information (e.g., project name and dataset name) to potentially trigger memorization. For the prompt details in MemPrompt, please refer to the left part in Fig. 2. Under the null hypothesis H_0 , this project and dataset names should not be helpful for the repair process.

For the benchmarks, we selected two APR benchmarks: Defects4J 1.2 [27] and GitBug-Java [43]. Defects4J is a widely used dataset that has been available for many years, making it highly likely to have been included in the training data of the two selected LLMs. In contrast, GitBug-Java is a very recent dataset, established after the emergence of these two LLMs, making it unlikely to be part of their training data. By comparing the results on a potentially memorized dataset (Defects4J) with those on an unmemorized dataset (GitBug-Java), we can gain deeper insights into memorization issues and assess the effectiveness of the detection methods.

Using ChatRepair-Single and MemPrompt, we repaired bugs from the selected datasets and collected the corresponding correct fixes, which are our primary focus. Our objective is to determine to what extent these correct fixes are a result of memorization.

In the study, we aim to answer the three research questions:

- **RQ1.** *How effectively do LLM-based APR tools perform on established benchmarks?* Specifically, how closely do these bug fixes align with the ground-truth fixes provided by developers?
- **RQ2.** *How do the bug fixes change when we add mutations into the buggy code?* This investigates the LLM-based repairs when they do not have enough context or the variable names are changed.
- **RQ3.** *How effective are current methodologies in detecting memorization within LLM-based program repair?* This evaluates the reliability of existing approaches in identifying whether fixes are based on simple memorization.

4.1 Experimental Setup

4.1.1 RQ1. How effectively do LLM-based APR tools perform on established benchmarks?

In this experiment, our goal is to assess the effectiveness of current LLM-based APR tools, specifically ChatRepair-Single and MemPrompt, and to analyze potential memorization issues in their generated fixes using the basic ground-truth matching method. For each model, we experiment with various temperature settings, specifically 0.0, 0.5, 0.8, and 1.0, to generate patches. Given the inherent randomness in LLM outputs, we attempt to repair each bug 30 times to capture a range of possible solutions, adopting the Pass@ k metric where $k = 30$ represents the number of trials. For those fixes that are correct, we subsequently check the extent to which the generated fixes exactly replicate the ground-truth solutions provided by developers.

4.1.2 RQ2. How do the bug fixes change when we add mutations to the buggy code?

This experiment studies the memorization issue with the mutation-based method, which is applied specifically to the subset of bugs that were successfully fixed in RQ1. Due to resource constraints, we utilize only MemPrompt for this part of the study, as it is better to indicate memorization effects, i.e., a lower probability event by removing feedback information compared to ChatRepair-Single.

Table 2. Repair performance of different LLMs on Defects4J and GitBug-Java. *Same_Rate* denotes the proportion of correctly repaired cases that achieve an exact match. *Same_Rate-L* refers to the proportion of exact-match among correctly repaired cases having large patch space, while *Same_Rate-S* represents the proportion of those cases with small patch space.

GitBug-Java(68)	Tool	Temperature	#Plausible	#Correct	#Same	Same_Rate	Correct_Rate	#Correct-L	#Same-L	Same_Rate-L	#Correct-S	#Same-S	Same_Rate-S
GPT-3.5	ChatRepair-Single	1	32	25	16	64.00%	36.76%	10	2	20.00%	15	14	93.33%
		0.8	28	21	16	76.19%	30.88%	7	3	42.86%	14	13	92.86%
		0.5	26	24	13	56.52%	35.29%	11	2	18.18%	13	11	84.62%
		0	10	10	3	30.00%	14.71%	6	0	0.00%	4	3	75.00%
		1	17	14	7	50.00%	20.59%	5	0	0.00%	9	7	77.78%
		0.8	16	12	7	58.33%	17.65%	4	1	25.00%	8	6	75.00%
	MemPrompt	0.5	15	13	8	61.54%	19.40%	4	1	25.00%	9	7	77.78%
		0	2	1	1	100.00%	1.47%	0	0	N/A	1	1	100.00%
		1	24	20	11	55.00%	29.41%	7	1	14.29%	13	10	76.92%
		0.8	21	17	10	58.82%	25.00%	7	1	14.29%	10	9	90.00%
		0.5	21	18	7	38.89%	26.47%	8	0	0.00%	10	7	70.00%
		0	6	6	6	100.00%	8.83%	0	0	N/A	6	6	100.00%
CodeLlama	ChatRepair-Single	1	15	11	7	63.64%	16.18%	3	0	0.00%	8	7	87.50%
		0.8	16	10	6	60.00%	14.71%	2	0	0.00%	8	6	75.00%
		0.5	14	11	7	63.64%	16.18%	3	0	0.00%	8	7	87.50%
		0	2	2	2	100.00%	2.94%	0	0	N/A	2	2	100.00%
	MemPrompt	1	82	59	50	84.75%	38.31%	34	26	76.47%	25	24	96.00%
		0.8	80	58	42	72.41%	37.66%	33	21	63.64%	25	21	84.00%
		0.5	77	57	46	80.70%	37.01%	32	23	71.88%	25	23	92.00%
		0	26	18	14	77.78%	11.69%	9	5	55.56%	9	9	100.00%
		1	60	44	31	70.45%	28.57%	19	10	52.63%	25	21	84.00%
		0.8	55	41	31	75.61%	26.62%	18	12	66.67%	23	19	82.61%
Defects4J(154)	GPT-3.5	0.5	59	41	32	78.05%	26.62%	20	13	65.00%	21	19	90.48%
		0	13	11	10	90.91%	7.14%	2	2	100.00%	9	8	88.89%
		1	71	50	39	78.00%	32.47%	22	14	63.64%	28	25	89.29%
	MemPrompt	0.8	68	49	45	91.84%	31.82%	22	19	86.36%	27	26	96.30%
		0.5	62	45	34	75.56%	29.22%	22	13	59.10%	23	21	91.30%
		0	2	2	1	100.00%	0.65%	0	0	N/A	1	1	100.00%
GitBug-Java(68)	ChatRepair-Single	1	56	40	33	82.50%	25.97%	15	9	60.00%	25	24	96.00%
		0.8	52	36	31	86.11%	23.38%	12	9	75.00%	24	22	91.67%
		0.5	55	34	29	85.29%	22.08%	12	8	66.67%	22	21	95.45%
		0	10	4	4	100.00%	2.60%	0	0	N/A	4	4	100.00%
	MemPrompt	1	56	40	33	82.50%	25.97%	15	9	60.00%	25	24	96.00%
		0.8	52	36	31	86.11%	23.38%	12	9	75.00%	24	22	91.67%

We modify the buggy code by introducing mutations and then observe whether the fixes generated under these altered conditions still align with the original repairs.

4.1.3 RQ3. How effective are current methodologies in detecting memorization?

This experiment aims to evaluate the effectiveness of existing methods for detecting memorization issues in LLM outputs. A major challenge in this evaluation is the absence of a labeled dataset specifically identifying memorized and non-memorized cases (i.e., ground truth). To address this, we use our memorization results from RQ2 to construct such a dataset. We believe the second method (i.e., mutation-based evaluation) serves as a very strong indicator of memorization, as it is nearly impossible to generate an exact-matching patch without the necessary variables and context under H_0 . Specifically, we label the memorized cases from Defects4J, detected in RQ2, as memorized examples. For non-memorization cases, we collect all correct fixes which not exactly match the ground truths from the GitBug-Java dataset, since this benchmark was created after the release of the selected LLMs, making memorization unlikely.

To assess the performance of the detection methods, we use the Area Under the Receiver Operating Characteristic (AUROC) metric. We selected several baselines that are applicable in our scenario (see Table 1), including Perplexity [13, 29], Min-K% [41], CDD [15], PPL-zlib ratio [12], and PPL-PPL ratio [53].

5 Evaluation Results

5.1 RQ1. Ground-truth Matching Analysis

We evaluate the performance of ChatRepair-Single and MemPrompt with different temperature settings on Defects4J and Gitbug-Java. The results are shown in Table 2, where column *#Plausible* shows the number of fixes that can pass all tests (i.e., plausible patches), the column *#Correct*

shows the number of correct fixes, the column *#Same* denotes the number of patches that are identical to the ground-truth in the benchmarks, the column *Same_Rate* is the proportion of identical generations out of the correct generations in percentage, and the column *Correct_Rate* represents the proportion of correctly fixed bugs out of the total number of bugs in the benchmark.

Note that the low-probability property is crucial in the detection framework. For exact matches, the probability is low when the patch space is large. An exception arises when the patch space is small, making it relatively more likely to obtain an exact match (e.g., when only a comparison operator needs to be changed). In these higher-probability cases, we cannot reliably determine whether memorization has occurred. However, automatically calculating the patch space is challenging due to the complex code structures. To address this, we manually reviewed all correctly fixed bugs, dividing them into high-probability cases (i.e., small patch space), denoted as *Correct-S*, and low-probability cases (i.e., larger patch space), denoted as *Correct-L*.

Specifically, we categorize bugs as *Correct-S* if: 1) the buggy code snippet contains only a single line; 2) the patch does not include identifiers not present in the context; or 3) the fix involves modifying only a few operators or operands. Bugs in the *Correct-S* and *Correct-L* categories that exhibit an exact match with the ground truth are further labeled as *Same-S* and *Same-L*, with their proportions represented as *Same_Rate-S* and *Same_Rate-L*.

From the results, we can observe that the number of correct fixes is significantly lower than the plausible fixes on both models and datasets, which indicates that for some bugs, although the model-generated results can pass the test case verification, manual verification still reveals that they are still not the correct repair solutions. In addition, the number of correct fixes increases with the increase in temperature value. We speculate that as the temperature increases, the model's output becomes more diverse, and this increased diversity in the model's outputs is helpful for program repair. When the temperature is set to 1.0, the two tools with both models have the highest number of fixes. Therefore, we use this setting in the subsequent experiments.

For the memorization, we can find that there are many correct repairs that are exactly the same as the ground truth provided in the dataset. Specifically, on the dataset Defects4J, the average *same_rate* of GPT-3.5 and CodeLlama across different tools and temperature settings are 78.83% and 87.42%, respectively. While the average *same_rate* of GPT-3.5 and CodeLlama on GitBug-Java are 62.07% and 67.50%. The results suggest that the LLMs demonstrate a diminished level of memorization for the newly constructed dataset, which has less probability to see this dataset, resulting in a smaller proportion of repairs that perfectly align with the ground-truths.

However, the *same_rate* in GitBug-Java exceeds 60%, which is still surprising. Our in-depth analysis reveals that most of the exact matches come from higher-probability cases (i.e., *same_rate-S*). For instance, the average *same_rate-S* for GPT-3.5 and CodeLlama across different tools and temperature settings is 84.55% and 85.87% on GitBug-Java, and 89.75% and 95.00% on Defects4J. These results, however, cannot be used as indicators of memorization. In the lower-probability cases, the average *same_rate-L* for the two models is 16.38% and 3.57% on GitBug-Java, indicating that exact matches are rare when the data is unlikely to have been memorized.

Low Probability of the Exact Match Event. We distinguish between exact matches for large code pieces (*same_rate-L*) and small code pieces (*same_rate-S*). Only *same_rate-L* is used to discuss memorization, as it represents a lower-probability event. The average *same_rate-L* for the two models is 68.98% and 51.35% on Defects4J. By comparing the *same_rate-L* results between Defects4J and GitBug-Java, we demonstrate that the exact match is a valid low-probability event under H_0 , i.e., *if the data has not been memorized, the likelihood of producing an exact match is comparatively low* (e.g., 16.38% and 3.57% on GitBug-Java). Specifically, considering the numerous possibilities for repairing buggy programs within *same_rate-L* (i.e., large patch space), including various semantically equivalent programs with different coding styles or syntaxes, exact matches

Table 3. Results of bug fixes after mutating the buggy programs. Patches in both *Type-1* and *Type-2* are considered as exact-match (*#Same*).

GitBug-Java(68)	Mutation Strategy	#Plausible	#Correct	#Type-1	#Type-2	#Type-3	#Type-4	#Same (Type-1&Type-2)
GPT-3.5 (14)	Mask	10	6	2 (33.33%)	0 (0%)	0 (0%)	0 (0%)	2 (33.33%)
	Omit	8	6	2 (33.33%)	0 (0%)	0 (0%)	0 (0%)	2 (33.33%)
	Rename	12	11	0	0	0	0	0
	Swap	12	10	0	0	0	0	0
CodeLlama (11)	Mask	11	7	2 (28.57%)	0 (0%)	0 (0%)	0 (0%)	2 (28.57%)
	Omit	9	7	2 (28.57%)	0 (0%)	0 (0%)	0 (0%)	2 (28.57%)
	Rename	9	9	0	0	0	0	0
	Swap	10	9	0	0	0	0	0
Defects4J(154)	Mutation Strategy	#Plausible	#Correct	#Type-1	#Type-2	#Type-3	#Type-4	#Same (Type-1&Type-2)
GPT-3.5 (44)	Mask	27	14	8 (57.14%)	2 (14.29%)	3 (21.43%)	1 (7.14%)	10 (71.43%)
	Omit	22	11	8 (72.73%)	1 (9.09%)	1 (9.09%)	1 (9.09%)	9 (81.82%)
	Rename	28	23	3	1	0	0	4
	Swap	23	20	1	4	0	0	5
CodeLlama (40)	Mask	27	23	18 (78.26%)	1 (4.35%)	1 (4.35%)	3 (13.04%)	19 (82.61%)
	Omit	28	17	14 (82.35%)	1 (5.88%)	0 (0%)	2 (11.76%)	15 (88.24%)
	Rename	27	24	1	1	0	0	2
	Swap	28	22	1	0	0	0	1

should be rare if no memorization has occurred. In contrast, the significantly higher *same_rate-L* on Defects4J (i.e., 68.98% and 51.35%) suggests that the correctly fixed bugs in Defects4J are more likely to result from memorization.

Furthermore, the repair accuracy of MemPrompt is generally lower than that of ChatRepair-Single, as it omits critical feedback information, such as error messages, from the prompts provided to the LLMs. This omission decreases the probability of generating a correct repair and achieving an exact match. Nevertheless, the performance of MemPrompt still remains surprisingly competitive. In some settings of Defects4J, the rates of exact matches produced by MemPrompt even surpass those of ChatRepair-Single, for instance, at $T=0$ and $T=0.8$ under GPT-3.5, as well as $T=0.5$ and $T=1.0$ under CodeLlama. These outcomes (i.e., both the competitive correct rates and the higher rates of exact matches) of MemPrompt further heighten suspicions that these fixes might be memorized.

In addition, compared to the repair performance between the two models, we can observe that GPT-3.5 performs better than CodeLlama across different settings. Further analysis of the average *same_rate* on Defects4J between ChatRepair-Single and MemPrompt across different temperature settings on GPT-3.5 and CodeLlama reveals notable differences. For GPT-3.5, the average *same_rate* are 78.91% for ChatRepair-Single and 78.76% for MemPrompt. In contrast, for CodeLlama, these rates increase to 86.35% and 88.48%, respectively.

Answer to RQ1: The results indicate that many of the correct repairs on Defects4J may be attributed to potential memorization (e.g., an average of 68.98% and 51.35% exact matches in large patch space cases, *same_rate-L*). In contrast, the significantly lower exact match results in the unlikely memorized dataset GitBug-Java (e.g., an average of 16.38% with GPT-3.5 and 3.57% with CodeLlama in *same_rate-L*) further demonstrate the validity of the ground-truth matching approach outlined in our hypothesis testing framework.

5.2 RQ2. Mutation-Based Memorization Analysis

While the findings from RQ1 raise significant concerns about memorization, we conducted a further analysis using mutation-based reassessment, which serves as a stronger indicator of memorization. Due to resource constraints, we primarily utilized MemPrompt for this analysis. As detailed in Table 2, we selected 44 and 40 bugs on Defects4J that were successfully repaired by MemPrompt with GPT-3.5 and CodeLlama, respectively, under optimal settings for further examination. Similarly, we also selected 14 and 11 bugs on GitBug-Java. Using the mutation strategies outlined in Section 3.3.2, we mutated the buggy functions and reapplied MemPrompt for repair, subsequently comparing the new patches to the original ones.

The results, presented in Table 3, show a significant drop in the success rates after applying large mutations. Specifically, for GPT-3.5 and CodeLlama, only 23 (52.27%) and 24 (60%) of the originally correct cases were successfully repaired after the mutations. This decrease observed with the Omit and Mask strategies indicates that the successful repair of certain bugs heavily depends on contextual information. The drop in performance with the Rename and Swap strategies suggests that the models lack robustness, implying they may not truly understand the bugs and repairs, but instead perform well primarily on the original versions, which could be potentially memorized. For example, after simple variable name swaps that do not alter the semantics, we observed that only 20 (45.45%) and 22 (55.00%) bugs were correctly repaired, suggesting that the models have not effectively understood the underlying semantics of the bug fixes and may be relying on memorization.

However, on GitBug-Java, we observed that the number of successfully repaired bugs using the Rename and Swap strategies did not decrease significantly compared to Defects4J. This is because that the likelihood of memorization for correctly fixed bugs on GitBug-Java is much lower, and the correct fixes primarily rely on comprehension rather than memorization. As a result, most bugs could be still successfully repaired even after variable-level perturbations.

Considering the low-probability event of comparing the new patches (on mutated buggy programs) with the ground-truth fixes of the original buggy programs, we surprisingly find a significant portion of matched cases in Defects4J. Specifically, we categorize the matching into four categories:

- *Type-1*: The patch is exactly the same to the ground truth.
- *Type-2*: The patch includes the ground truth but repeats some lines from the original bug.
- *Type-3*: The patch is identical to the ground truth except for some minor differences in the control flow structure.
- *Type-4*: The patch is different from the ground truth, but they are semantic equivalent.

For the Mask and Omit strategies, we further computed the proportion of matched bugs within each category out of all the correctly fixed cases (with the percentage results shown in brackets). We did not calculate percentages for the Rename and Swap strategies since the ground truth has been altered compared to the original bugs. If the generated fix matches the original ground truth in these cases, it cannot be considered a correct fix.

Considering the results under various mutation types, we observe that Type-1 (exact-match) under the Mask and Omit mutations constitutes a significant portion of the successful repairs. We analyze Type-1 and Type-2 combined (as shown in the column *#Same (Type-1&Type-2)*) because both types result in exact-match patches with the original ground truth, although Type-2 includes additional outputs. These combined types account for a substantial fraction of the correct fixes, especially in Defects4J. For instance, under GPT-3.5, Type-1&Type-2 accounts for 10 (71.42%) and 9 (81.82%), respectively. Similarly, under CodeLlama, Type-1&Type-2 represents 19 (82.61%) and 15 (88.24%), respectively. These findings highlight that, despite the lack of necessary information following Mask and Omit mutations, most successful repairs closely replicate the ground-truth patches, which strongly suggests potential memorization. The proportion of exact matches on GitBug-Java is 33.33% and 28.57% for the two models, respectively. While these percentages may seem high, the absolute number of cases is still low (i.e., 2), indicating that the occurrence of the event is much rarer compared to bugs in Defects4J.

Regarding the Rename and Swap operations, the occurrence of Type-1&Type-2 is significantly lower compared to Mask and Omit, which is expected, as these mutations alter the variable names, thus changing the ground truth of the patches. In fact, the probability of an exact match in this scenario should be the lowest, making it the strongest indicator of memorization. However, there are still some instances of Type-1&Type-2 cases observed in Defects4J. For example, under GPT-3.5,

4 and 5 patches continue to use the names from the original buggy programs. Under CodeLlama, 2 and 1 patches revert to using the names from the original buggy programs. We also find that on GitBug-Java, no one exactly matched fix are obtained through these two strategies.

Low Probability of the Exact Match Event after Large Mutation. The significant difference between the two datasets (i.e., Defects4J and GitBug-Java) highlights the validity of the designed mutation-based approach in our hypothesis testing framework. Specifically, *if the data has not been memorized (e.g., in GitBug-Java), the likelihood of producing an exact match is comparatively low (e.g., 0 in GitBug-Java after Rename and Swap).*

Comparison between Two Memorization Analysis Methods. We further compare the memorization results using the two different methods discussed in RQ1 and RQ2. Specifically, we focus on the correct fixes in Defects4J (i.e., 44 bugs in GPT-3.5 and 40 in CodeLlama) achieved by MemPrompt, as this method was used in RQ2. For these correctly fixed bugs, we examined the overlap (14 for GPT-3.5 and 16 for CodeLlama) of potential memorized cases identified by the two methods. The mutation-based analysis detects fewer unique cases, e.g., 3 for both GPT-3.5 and CodeLlama. In contrast, the basic ground-truth matching identified a higher number of unique cases (e.g., 17 for both GPT-3.5 and CodeLlama). These findings demonstrate that the mutation-based method is a stronger indicator, as the probability of such events occurring is much lower than that of an exact ground-truth match under the null hypothesis (H_0). When such a mutation-based event occurs, it provides high confidence in confirming memorization.

Answer to RQ2: The mutation-based method strongly substantiates the memorization issues in LLM-based repairs. In these mutation scenarios, it is expected to be nearly impossible to generate exact-match cases. The comparative results between Defects4J and GitBug-Java highlight the low probability of mutation-based events under the null hypothesis (H_0), which is crucial in the hypothesis testing framework. Additionally, the comparison between RQ1 and RQ2 demonstrates that the mutation-based method is a stronger indicator of memorization, as it has a lower probability of occurrence than basic ground-truth matching.

5.3 RQ3. Evaluation of Existing Memorization Detection Methods

To evaluate the effectiveness of existing memorization analysis methods in the LLM-based APR task, we constructed a dataset as described in Section 4.1.3, which includes cases of memorization and non-memorization derived from the findings of RQ2. The memorization cases encompass all the correctly fixed bugs in Defects4J whose patches exactly match the ground-truths (T1&T2) after Mask and Omit mutations (e.g., 10 and 9 for GPT-3.5, 19 and 15 for CodeLlama), as well as those kind of bugs following Rename and Swap mutations (4 and 5 for GPT-3.5, 2 and 1 for CodeLlama). We categorized other correctly fixed bugs in GitBug-Java whose patches are not the same as the ground-truths as non-memorization cases. Subsequently, we calculated the AUROC score for various metrics, configuring Min-K% and CDD with different parameters. Note that the LLM output may include additional information beyond the patch itself (e.g., explanations). Therefore, we evaluate these tools in two scenarios: using the entire LLM output (Entire) and only the patch portion (Patch).

Table 4 shows the performance with different configurations. Note that Perplexity and Min-K% metrics are not applicable to GPT-3.5, as they require model outputs that are not available from ChatGPT, and we use ‘-’ instead. As shown in Table 4, the AUROC values for all configurations, including those using the entire output and the patch only, are notably low, with most falling below 0.5. Min-K% demonstrated marginally better performance than Perplexity, with Min-5% achieving a maximum value of 0.5460 under the Entire configuration. However, in the majority of the settings, its value did not exceed 0.5. The best-performing metric is CDD, with values exceeding 0.5 across

Table 4. Results of memorization detection by existing works

Type	Metric	Model		Type	Metric	Model	
		GPT-3.5	CodeLlama			GPT-3.5	CodeLlama
Entire	Perplexity	-	0.4216	Patch	Perplexity	-	0.3986
	Min-5%	-	0.5460		Min-5%	-	0.3865
	Min-10%	-	0.5219		Min-10%	-	0.4041
	Min-20%	-	0.4726		Min-20%	-	0.4070
	Min-30%	-	0.4526		Min-30%	-	0.4029
	Min-40%	-	0.4397		Min-40%	-	0.4007
	Min-50%	-	0.4272		Min-50%	-	0.3988
	Min-60%	-	0.4243		Min-60%	-	0.4010
	CDD ($\alpha = 0.10$)	0.5000	0.4545		CDD ($\alpha = 0.10$)	0.6255	0.5878
	CDD ($\alpha = 0.15$)	0.5000	0.5165		CDD ($\alpha = 0.15$)	0.6255	0.5223
	CDD ($\alpha = 0.20$)	0.5789	0.5145		CDD ($\alpha = 0.20$)	0.6113	0.5908
	PPL-zlib ratio	-	0.4493		PPL-zlib ratio	-	0.4703
	PPL-PPL ratio	-	0.3481		PPL-PPL ratio	-	0.3697

Table 5. Probability of event occurrence and corresponding thresholds

Scores and Probability	Perplexity	Min -5%	Min -10%	Min -20%	Min -30%	Min -40%	Min -50%	Min -60%	PPL-PPL ratio	PPL-zlib ratio	CDD ($\alpha = 0.20$)
Threshold (Defects4J)	0.082	0.949	0.781	0.448	0.289	0.215	0.167	0.141	1.843	0.0012	0.611
Avg.Score (GitBug-Java)	0.119	1.308	0.994	0.587	0.416	0.309	0.242	0.205	2.486	0.0015	0.574
Occurrence Probability	51.3%	62.5%	55.0%	47.5%	48.8%	48.8%	47.5%	47.5%	57.5%	67.5%	58.3%

all settings. Notably, CDD ($\alpha = 0.10$) and CDD ($\alpha = 0.15$) achieved the highest score 0.6255 under the Patch configuration. This indicates that the current methods are ineffective in reliably detecting memorization cases within the APR task.

Event Probability Analysis. As discussed in Section 3.1, the key requirement for identifying memorization is the low-probability of the event. To investigate the reasons behind the poor results of existing methods, we estimated the probability of these events (see Table 1) under the null hypothesis. The challenge lies in determining whether an event (e.g., a Perplexity value below a certain threshold) occurs. To define this threshold, we used the average score of each method on the memorized cases (i.e., from Defects4J), as shown in Row *Threshold (Defects4J)* of Table 5.

Intuitively, if the values from the corresponding metrics are less than or greater than these average scores, the low-probability event is considered to have occurred. We then used these thresholds to calculate the probability of the event occurring on non-memorized cases from GitBug-Java (i.e., under the null hypothesis). Row *Occurrence Probability* presents these probability results, which indicate that the probability of these events occurring under H_0 is quite high (47.5%-67.5%), meaning that even in unmemorized cases, the probability remains close to or exceeds 50%, failing to meet the low-probability requirements in our framework (see the theoretical analysis in Section 3.1). As a result, these methods perform poorly. For comparison, we also present the average scores on non-memorized cases in Row *Avg.Score (GitBug-Java)*. We observe that in most cases, there is no significant difference between the average scores of memorized and non-memorized cases.

In contrast, the probability of events occurring under the null hypothesis in our method is much lower (see the discussion in RQ1 and RQ2), particularly under the Rename and Swap strategies, where the probability is 0. These results underscore the importance of our framework and highlight the need for well-designed memorization detection events and their proper validation.

Answer to RQ3: The current memorization analysis methods are ineffective at identifying memorization cases within the program repair task because the designed events do not adequately meet the low-probability requirements of the hypothesis testing framework.

Table 6. Results of GPT-3.5 on advanced prompting strategy

Mutation Strategy	Prompting Strategy	#Plausible	#Correct	#Type-1	#Type-2	#Type-3	#Type-4	#Same(Type-1&Type-2)
Defects4J-Rename (44)	MemPrompt	28	23	3	1	0	0	4
	ChatRepair-Single	30	27	0	2	0	0	2
	ChatRepair-Single+CoT	34	32	0	0	0	0	0
	ChatRepair-Single+Few	28	27	0	0	0	0	0
Defects4J-Swap (44)	MemPrompt	23	20	1	4	0	0	5
	ChatRepair-Single	28	23	2	2	0	0	4
	ChatRepair-Single+CoT	29	27	0	2	0	0	2
	ChatRepair-Single+Few	28	25	0	1	0	0	1

6 Discussion

6.1 Reasoning and Memorization

An intriguing question is the relationship between reasoning (i.e., true understanding) and memorization, as both can influence model outputs. To explore this, we conducted additional experiments to investigate how enhancing reasoning capabilities affects model memorization. Specifically, two key factors contribute to improved reasoning: the use of advanced prompting strategies and scaling up to larger models with inherently stronger reasoning abilities.

Memorization with Advanced Prompts. To examine the impact of advanced prompts on memorization, we incorporated advanced prompting techniques, including Chain-of-Thought (CoT) [54] and Few-Shot prompting [52]. Building upon the experimental setup of RQ2, we extended our analysis by evaluating GPT-3.5 on Defects4J.

As shown in Table 6, models utilizing advanced prompts exhibited enhanced reasoning capabilities, which in turn led to a significant reduction in memorization. For GPT-3.5, under the Rename strategy, no ground-truth matches were observed. Similarly, under the Swap strategy, integrating CoT and Few-Shot prompting in ChatRepair-Single reduced the number of Type-1 and Type-2 memorization cases from 5 to 2 and 1, respectively. These results highlight the potential of reasoning-enhanced prompting techniques in mitigating memorization while improving the model's overall performance.

Memorization with Powerful LLMs. To investigate the impact of advanced LLMs on memorization, we expanded our analysis to include a broader range of models. Following the experimental setup in RQ2, we evaluated GPT-4o-mini, DeepSeek-Coder-16b, and Grok using the Rename and Swap strategies. The results are summarized in Table 7. Notably, all models except Grok+CoT and Grok+Few were tested using basic prompts.

While larger models generally demonstrated stronger reasoning and repair capabilities, their levels of memorization varied. Our findings indicate that all models exhibit some degree of memorization, with GPT-4o-mini showing the least and Grok displaying the most. Specifically, Grok produced 11 ground-truth matches across all strategies, despite achieving the best repair performance. Given this high number of memorized cases (7+4) with the basic prompt, we further tested Grok with advanced prompts (Grok-CoT and Grok-Few). The results showed a significant reduction in memorization, with only 3 (2+1) cases observed under both CoT and Few-Shot prompting. These findings are consistent with our observations in Table 6.

Based on the results above, we have the following key findings:

- Larger models tend to exhibit both stronger reasoning and higher memorization tendencies. For example, Grok with basic prompts demonstrates notable performance and memorization effects.
- Advanced prompting techniques enhance reasoning capabilities, thereby reducing reliance on memorization in predictions.
- Memorization can be problematic, particularly when the expected answer differs from the memorized one. It is crucial to distinguish whether a model's predictions stem primarily from reasoning or memorization to ensure reliability and generalizability.

Table 7. Results on larger LLMs

Mutation Strategy	Model	#Plausible	#Correct	#Type-1	#Type-2	#Type-3	#Type-4	#Same(Type-1&Type-2)
Defects4J-Rename (44)	GPT-3.5	30	27	0	2	0	0	2
	GPT-4o-mini	33	30	0	1	0	0	1
	DeepSeek-Coder-16b	32	28	1	1	0	0	2
	Grok	36	32	2	5	0	0	7
	Grok+CoT	35	32	1	1	0	0	2
	Grok+Few	38	35	1	1	0	0	2
Defects4J-Swap (44)	GPT-3.5	28	23	2	2	0	0	4
	GPT-4o-mini	30	27	0	1	0	0	1
	DeepSeek-Coder-16b	27	23	0	3	0	0	3
	Grok	32	28	1	3	0	0	4
	Grok+CoT	34	30	0	1	0	0	1
	Grok+Few	35	31	0	1	0	0	1

6.2 Evaluation Suggestions for LLM-Based SE Tasks

Our findings indicate that although current LLM-based repair methods significantly outperform traditional methods, a part of the solutions provided by LLMs tend to result from memorization rather than genuine understanding. Consequently, when evaluating LLM-based methods, it is crucial to distinguish between:

- *Base Models vs. Proposed Methods*: Identifying which cases are effectively addressed by the underlying LLMs rather than any enhancements introduced by new methods. This helps assess whether the improvements are due to the base model's capabilities or innovative strategies.
- *Memorization vs. Understanding*: Discerning which cases result mainly from memorization. In some instances, basic prompts may be just effective in inducing memorization of LLMs while advanced prompts can help enhance the model's reasoning capabilities.
- *Genuine Method Efficacy*: Determining which cases are genuinely solved by the proposed methods, where the methods enhance the LLMs' ability to understand and resolve the task.

To effectively distinguish these scenarios, we have some actionable recommendations: 1) Utilize recently released or curated benchmarks to minimize the risk of memorization by the model. 2) Use mutation-based strategies, such as those proposed in our work, on existing benchmarks for more evaluation. 3) Evaluate the explanations provided by LLMs for their repair, and check the logical consistency. While our research focuses on APR tasks, these conclusions are likely applicable to other software engineering tasks.

6.3 Memorization Detection

Determining memorization in LLM outputs is challenging. We developed a theoretical memorization detection framework based on hypothesis testing and proposed two methods as indicators of memorization. The probability of the selected event determines the confidence in confirming memorization. Although our mutation-based analysis is a stronger indicator due to its much lower probability, it is resource-intensive, requiring the re-running of mutated cases and post-analysis, such as equivalence comparisons between new and original outputs. This underscores the need for more lightweight and robust detection methods. The investigations in RQ3 reveal that existing methods fall short in accurately detecting memorization, particularly in LLM-based APR tasks, because these events may not be truly low-probability. Therefore, the probability of event occurrence in future memorization detection methods should be thoroughly validated before using them in memorization identification.

Another consideration is that the inclusion of data in the training set does not guarantee memorization. For instance, from RQ2, we observed that many Defects4J cases are likely memorized, indicating that the dataset appears to be included in the training data. However, the remaining cases in Defects4J cannot be detected as memorized by our two methods, suggesting that even if they were part of the training set, they may not have been memorized effectively.

7 Threats to Validity

The choice of benchmarks and LLMs poses the first threat to validity. We utilized GPT-3.5 and CodeLlama, which are commonly employed for code-related tasks, and Defects4J, a widely used dataset in program repair studies. However, our results could vary with different models or datasets. And our results could have shown memory problems and the need to detect them.

The selection of APR methods, such as ChatRepair and MemPrompt, constitutes the second threat. The APR methods employ various prompting strategies that may induce memorization to differing extents. We select two representative methods. ChatRepair provides more comprehensive guidance that aids in reasoning, whereas MemPrompt offers less, intentionally exposing the model to potential memorization. Although our primary aim is to uncover memorization issues, the choice of method could influence the extent of these issues.

The randomness of the LLM output could be a potential threat; we mitigated this by generating each solution 30 times. Another threat concerns the soundness of our memorization analysis due to the lack of ground-truth for memorization. To address this, we empirically demonstrated the low-probability property of our two methods based on our theoretical framework. Exact-match may not be as strong an indicator of memorization as mutation-based methods, given its relatively higher probability. However, to mitigate this limitation, we have focused our analysis on exact matches within a large patch space (`same_rate-L`), which represents a low-probability event. Additionally, we compared the probabilities across two different benchmarks, Defects4J and the unseen dataset `GitBug-Java`, to further demonstrate its low probability.

Another potential threat is the determination of high-probability and low-probability cases (i.e., `Same-S` and `Same-L`). Due to the absence of an automated method, we employed a conservative heuristic strategy for manual classification, which may introduce some inaccuracies. To enhance the reliability and transparency of our results, we plan to develop an effective metric for automatically estimating the potential patching space as part of our future work.

8 Related Work

8.1 LLM-Based Program Repair

LLM-based program repair aims to employ LLMs to repair program bugs. The majority of LLM-based APR methods fall into two categories, distinguished by their usage approach.

Conversation-Based Methods. It typically repairs bugs through conversations with the LLM and further provides it with program runtime feedback, such as information on test failures. Xia et al. [52] and Kong et al. [28] proposed the conversation-based program repair systems. However, due to the high randomness in the outputs of LLMs, ChatRepair [52] and ContrastRepair [28] require multiple attempts to achieve satisfactory repair performance, consuming significant resources in terms of time and money. To mitigate these problems, Hidvegi et al. [25] introduced CigaR, a token-efficiency LLM-based APR approach focused on minimizing the token costs associated with ChatGPT. Recently, some agent-based approaches have also been introduced, such as RepairAgent [8] treats ChatGPT as an autonomous agent capable of planning and executing actions to generate patches using dynamic prompts, thus enabling more adaptive and efficient problem-solving strategies.

Finetuning-Based Methods. There are also some fine-tuning based approaches for automated program repair [23, 42, 46, 55]. For example, Silva [42] escalated the parameters of fine-tuned LLMs to the billion scale and introduced RepairLLaMA based on CodeLlama-7B. RepairLLaMA employs a parameter-efficient fine-tuning technique called LoRA to train repair adapters, optimizing them for accurate code representation and repair.

Compared to existing studies, our work concentrates on elucidating the memorization issue within LLM-based program repair, thereby offering deeper insights for the development of more reliable program repair systems.

8.2 Dataset Contamination Detection

Dataset contamination is a prevalent issue in LLMs, gaining prominence as it can compromise benchmark evaluations if the evaluation examples have been inadvertently exposed during pre-training. Wei et al. [47] and Du et al. [16] define an example as contaminated if there is a 13-gram overlap between the training data and the evaluation example. Building on this, Chowdhery et al. [14] propose a more stringent criterion, considering an example contaminated if 70% of its 8-grams are found in the training data. However, these approaches necessitate access to the training corpora, which is often not available for the recently closed model. To address this limitation, alternative methods [1, 15, 20, 41] have been developed to detect contamination without direct access to the training data. For instance, Sainz et al. [1] employ a simple method where ChatGPT is prompted to generate examples by merely providing the name of the dataset, aiming to detect whether the model regurgitates data. Although these techniques achieve promising performance, they cannot apply into program repair, which inspires us proposing MemPrompt for research.

9 Conclusion

In this work, we proposed a theoretical framework for detecting memorization in LLM-based program repair, based on hypothesis testing. We further propose two methods to detect the memorization. Through comprehensive experiments on two distinct datasets, Defects4J and GitBug-Java, we demonstrated the effectiveness of our approach, particularly highlighting the robustness of mutation-based analysis as a lower-probability event for identifying memorization. Our findings reveal that a significant portion of correct repairs in Defects4J likely stem from memorization. Additionally, we showed that existing detection methods are inadequate for reliably identifying memorization in APR tasks. This study underscores the need to enhance evaluation methodologies for LLM-based research in software engineering tasks, with a particular consideration on the impact of memorization.

10 Data Availability

Our source code and experimental data are available at [4].

Acknowledgments

This research is supported by the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN), and the Lee Kong Chian Fellowship. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore.

References

- [1] 2023. Did chatgpt cheat on your test? <https://hitz-zentroa.github.io/lm-contamination/blog/>.
- [2] 2023. A Massively Spiffy Yet Delicately Unobtrusive Compression Library. <https://zlib.net/>
- [3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [4] Anonymous. 2024. MemPrompt. <https://sites.google.com/view/memprompt>.
- [5] Amitav Banerjee, UB Chitnis, SL Jadhav, JS Bhawalkar, and S Chaudhury. 2009. Hypothesis testing, type I and type II errors. *Industrial psychiatry journal* 18, 2 (2009), 127–131.

- [6] Susanne Bobzien. 2002. The development of *modus ponens* in antiquity: From Aristotle to the 2nd century AD. *Phronesis* 47, 4 (2002), 359–394.
- [7] Susanne Bobzien. 2006. Ancient logic. (2006).
- [8] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. *arXiv preprint arXiv:2403.17134* (2024).
- [9] Colin F Camerer and Howard Kunreuther. 1989. Decision processes for low probability events: Policy implications. *Journal of policy analysis and management* 8, 4 (1989), 565–592.
- [10] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramer, and Chiyuan Zhang. 2022. Quantifying memorization across neural language models. *arXiv preprint arXiv:2202.07646* (2022).
- [11] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. 2019. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX security symposium (USENIX security 19)*. 267–284.
- [12] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*. 2633–2650.
- [13] Stanley F Chen and Joshua Goodman. 1999. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language* 13, 4 (1999), 359–394.
- [14] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.
- [15] Yihong Dong, Xue Jiang, Huan Yu Liu, Zhi Jin, and Ge Li. 2024. Generalization or Memorization: Data Contamination and Trustworthy Evaluation for Large Language Models. *arXiv preprint arXiv:2402.15938* (2024).
- [16] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *International Conference on Machine Learning*. PMLR, 5547–5569.
- [17] Sarah Fakhoury, Aaditya Naik, Georgios Sakas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation. *arXiv preprint arXiv:2404.10100* (2024).
- [18] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.
- [19] Shahriar Golchin and Mihai Surdeanu. 2023. Data contamination quiz: A tool to detect and estimate contamination in large language models. *arXiv preprint arXiv:2311.06233* (2023).
- [20] Shahriar Golchin and Mihai Surdeanu. 2023. Time travel in llms: Tracing data contamination in large language models. *arXiv preprint arXiv:2308.08493* (2023).
- [21] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [22] Qi Guo, Xiaohong Li, Xiaofei Xie, Shangqing Liu, Ze Tang, Ruitao Feng, Junjie Wang, Jidong Ge, and Lei Bu. 2024. FT2Ra: A Fine-Tuning-Inspired Approach to Retrieval-Augmented Code Completion. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 313–324.
- [23] Sichong Hao, Xianjun Shi, Hongwei Liu, and Yanjun Shu. 2023. Enhancing Code Language Models for Program Repair by Curricular Fine-tuning Framework. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 136–146.
- [24] Valentin Hartmann, Anshuman Suri, Vincent Bindschaedler, David Evans, Shruti Tople, and Robert West. 2023. SoK: Memorization in General-Purpose Large Language Models. *arXiv preprint arXiv:2310.18362* (2023).
- [25] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. 2024. CigaR: Cost-efficient Program Repair with LLMs. *arXiv preprint arXiv:2402.06598* (2024).
- [26] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Inferfix: End-to-end program repair with llms. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1646–1656.
- [27] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
- [28] Jiaolong Kong, Mingfei Cheng, Xiaofei Xie, Shangqing Liu, Xiaoning Du, and Qi Guo. 2024. Contrastrepair: Enhancing conversation-based automated program repair via contrastive test case pairs. *arXiv preprint arXiv:2403.01971* (2024).
- [29] Yucheng Li. 2023. Estimating contamination via perplexity: Quantifying memorisation in language model evaluation. *arXiv preprint arXiv:2309.10677* (2023).

- [30] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, et al. 2022. Automating code review activities by large-scale pre-training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1035–1047.
- [31] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2024).
- [32] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. Tbar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*. 31–42.
- [33] Shangqing Liu, Yanzhou Li, Xiaofei Xie, Wei Ma, Guozhu Meng, and Yang Liu. 2024. Automated commit intelligence by pre-training. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–30.
- [34] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2024. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. *arXiv preprint arXiv:2401.08807* (2024).
- [35] Justus Mattern, Fatemehsadat Mireshghallah, Zhijing Jin, Bernhard Schölkopf, Mrinmaya Sachan, and Taylor Berg-Kirkpatrick. 2023. Membership inference attacks against language models via neighbourhood comparison. *arXiv preprint arXiv:2305.18462* (2023).
- [36] Gary H McClelland, William D Schulze, and Don L Coursey. 1993. Insurance for low-probability hazards: A bimodal response to unlikely events. *Making Decisions About Liability And Insurance: A Special Issue of the Journal of Risk and Uncertainty* (1993), 95–116.
- [37] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. 2023. Lever: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*. PMLR, 26106–26128.
- [38] Moumita Das Purba, Arpita Ghosh, Benjamin J Radford, and Bill Chu. 2023. Software vulnerability detection using large language models. In *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 112–119.
- [39] Md Rafiqul Islam Rabin, Aftab Hussain, Mohammad Amin Alipour, and Vincent J Hellendoorn. 2023. Memorization and generalization in neural code intelligence models. *Information and Software Technology* 153 (2023), 107066.
- [40] David Sanford. 2011. *If P, then Q: Conditionals and the Foundations of Reasoning*. Routledge.
- [41] Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2023. Detecting pretraining data from large language models. *arXiv preprint arXiv:2310.16789* (2023).
- [42] André Silva, Sen Fang, and Martin Monperrus. 2023. RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair. *arXiv preprint arXiv:2312.15698* (2023).
- [43] André Silva, Nuno Saavedra, and Martin Monperrus. 2024. GitBug-Java: A Reproducible Benchmark of Recent Java Bugs. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*. IEEE, 118–122.
- [44] Benjamin Steenhoeck, Hongyang Gao, and Wei Le. 2024. Dataflow Analysis-Inspired Deep Learning for Efficient Vulnerability Detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [45] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2024. Improving llm code generation with grammar augmentation. *arXiv preprint arXiv:2403.01632* (2024).
- [46] Weishi Wang, Yue Wang, Shafiq Joty, and Steven CH Hoi. 2023. Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 146–158.
- [47] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652* (2021).
- [48] Rand R Wilcox. 2011. *Introduction to robust estimation and hypothesis testing*. Academic press.
- [49] Chunqiu Steven Xia, Yifeng Ding, and Lingming Zhang. 2023. Revisiting the plastic surgery hypothesis via large language models. *arXiv preprint arXiv:2303.10494* (2023).
- [50] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
- [51] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971.
- [52] Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the Conversation Going: Fixing 162 out of 337 bugs for 0.42 each using ChatGPT. *arXiv preprint arXiv:2304.00385* (2023).
- [53] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, Donggyun Han, and David Lo. 2024. Unveiling memorization in code models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

- [54] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. Thinkrepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1274–1286.
- [55] Wei Yuan, Qunjun Zhang, Tiek He, Chunrong Fang, Nguyen Quoc Viet Hung, Xiaodong Hao, and Hongzhi Yin. 2022. CIRCLE: continual repair across programming languages. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 678–690.
- [56] Qunjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. Gamma: Revisiting template-based automated program repair via mask prediction. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 535–547.
- [57] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 47–51.

Received 2025-02-26; accepted 2025-04-01